Introduction to LLVM

(Low Level Virtual Machine)

Fons Rademakers
CERN

What is the LLVM Project?

- Collection of compiler technology components
  - Not a traditional "Virtual Machine"
  - Language independent optimizer and code generator
  - LLVM-GCC front-end
  - Clang front-end
  - Just-In-Time compiler
- Open source project with many contributors
  - Industry, research groups, individuals
  - See http://llvm.org/Users.html

- Everything is Open Source, and BSD Licensed! (except GCC)

- For more see: http://llvm.org/

LLVM as an Open-Source Project

- Abridged project history
  - Dec. 2000: Started as a research project at University of Illinois
  - Oct. 2003: Open Source LLVM 1.0 release
  - June 2005: Apple hires project lead Chris Lattner
  - June 2008: LLVM GCC 4.2 ships in Apple Xcode 3.1
  - Feb. 2009: LLVM 2.5

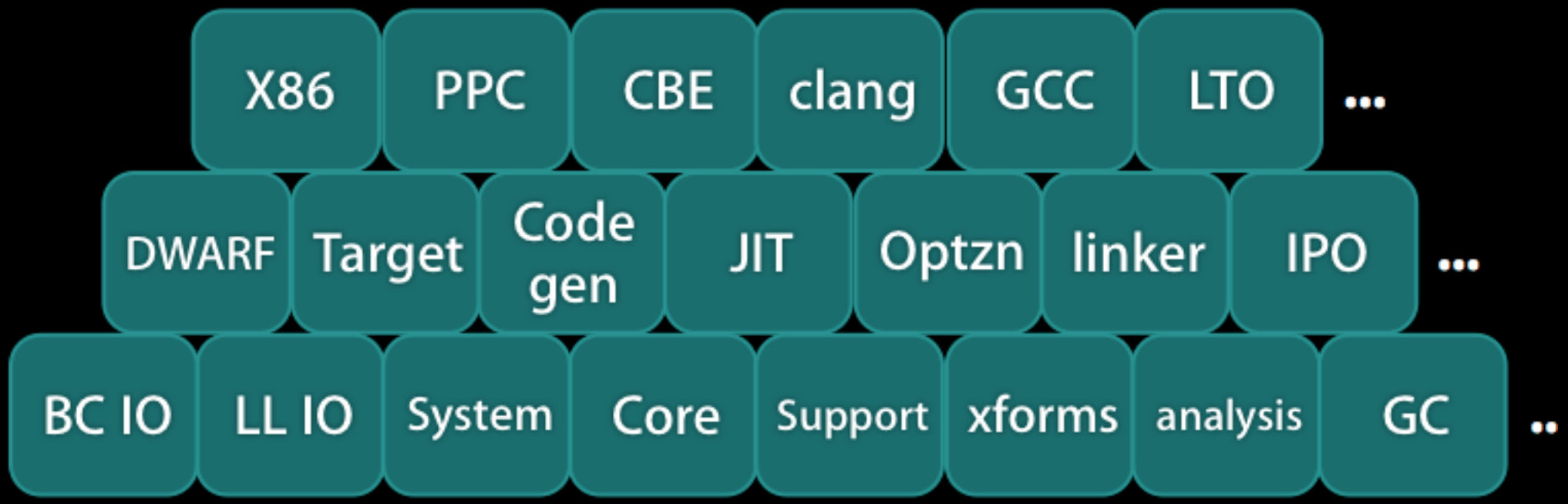
Why New Compilers?

- Existing Open Source C Compilers have Stagnated!

- How?
  - Based on decades old code generation technology
  - No modern techniques like cross-file optimization and JIT codegen
  - Aging code bases: difficult to learn, hard to change substantially
  - Can’t be reused in other applications
  - Keep getting slower with every release

LLVM Vision and Approach

- Build a set of modular compiler components:
  - Reduces the time & cost to construct a particular compiler
  - Components are shared across different compilers
  - Allows choice of the right component for the job
  - Focus on compile and execution times
  - All components written in C++, ported to many platforms

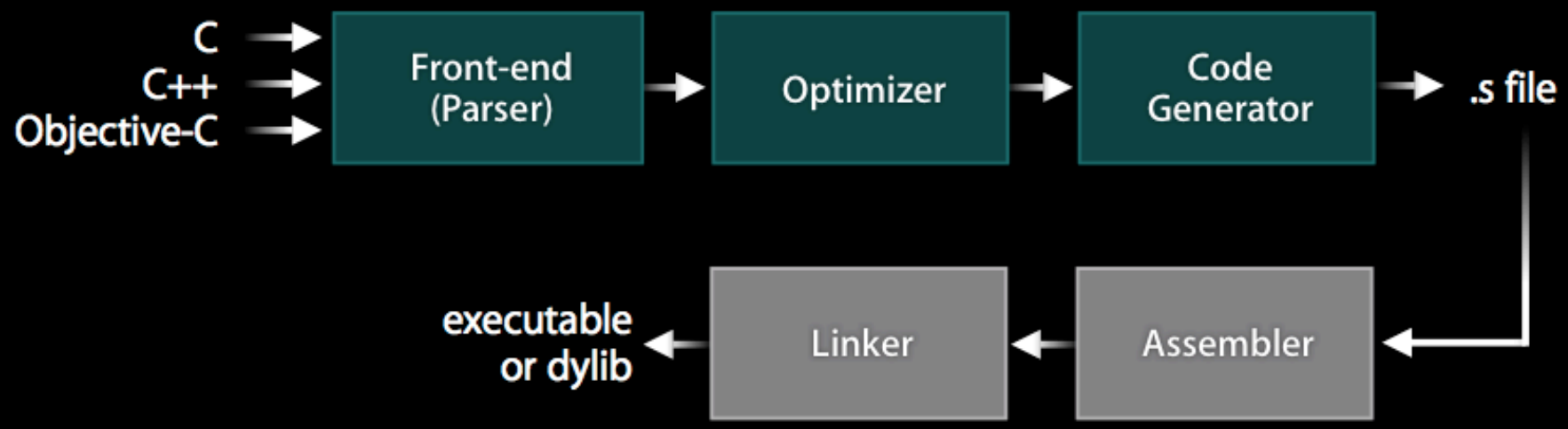


First Product - LLVM-GCC 4.2

- C, C++, Objective C, Objective C++, Ada and Fortran
- Same GCC command line options
- Supports almost all GCC language features and extensions
- Supports many targets, including X86, X86-64, PowerPC, etc.
- Extremely compatible with GCC 4.2
- Drop in replacement, compiles ROOT without problems

Standard Compiler Architecture

- Parser handles language-specific analysis
- Optimizers improve code
- Code generator maps onto processor
- Assembler and linker finish compilation

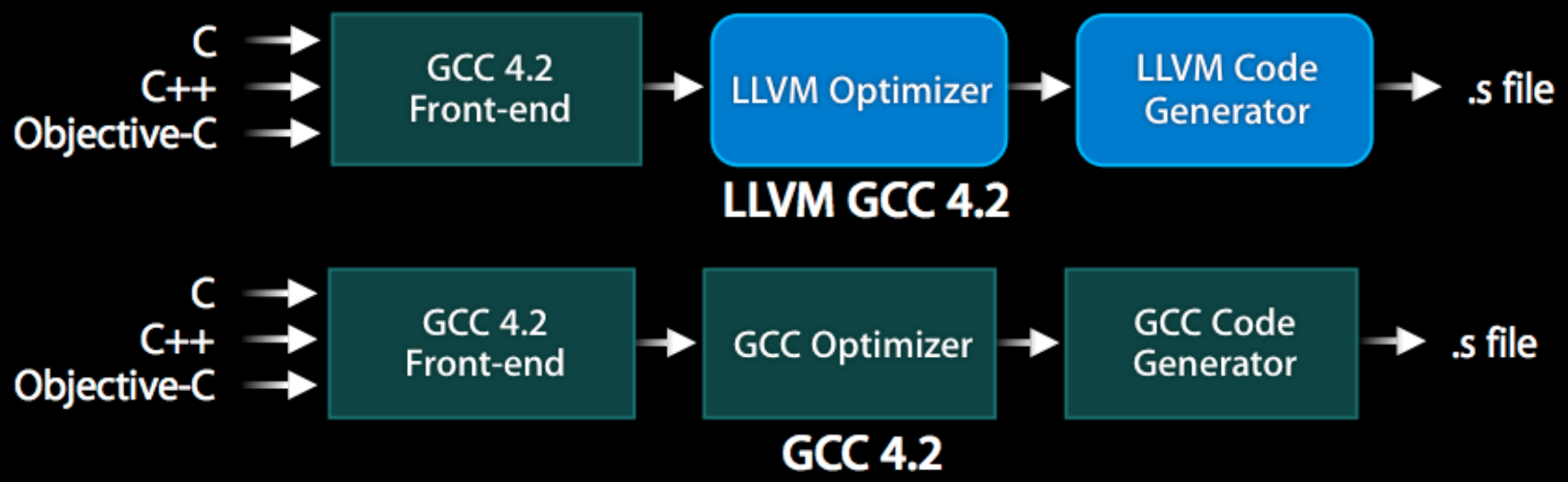


LLVM GCC 4.2 Design

- Replace GCC optimizer and code generator with LLVM
  - Reuses GCC parser and runtime libraries

LLVM GCC 4.2 Design

- Replace GCC optimizer and code generator with LLVM
  - Reuses GCC parser and runtime libraries



Linking LLVM and GCC Compiled Code

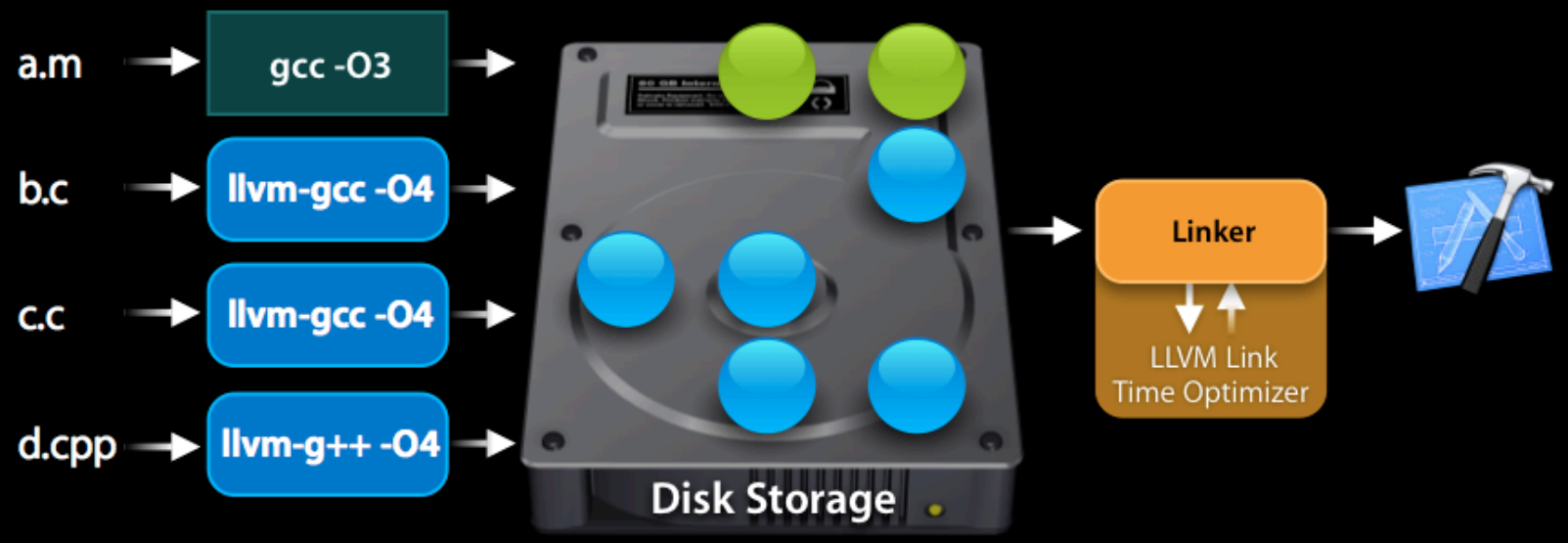
- Safe to mix and match .o files between compilers
- Safe to call into libraries built with other compilers

Potential Impact of LLVM Optimizer

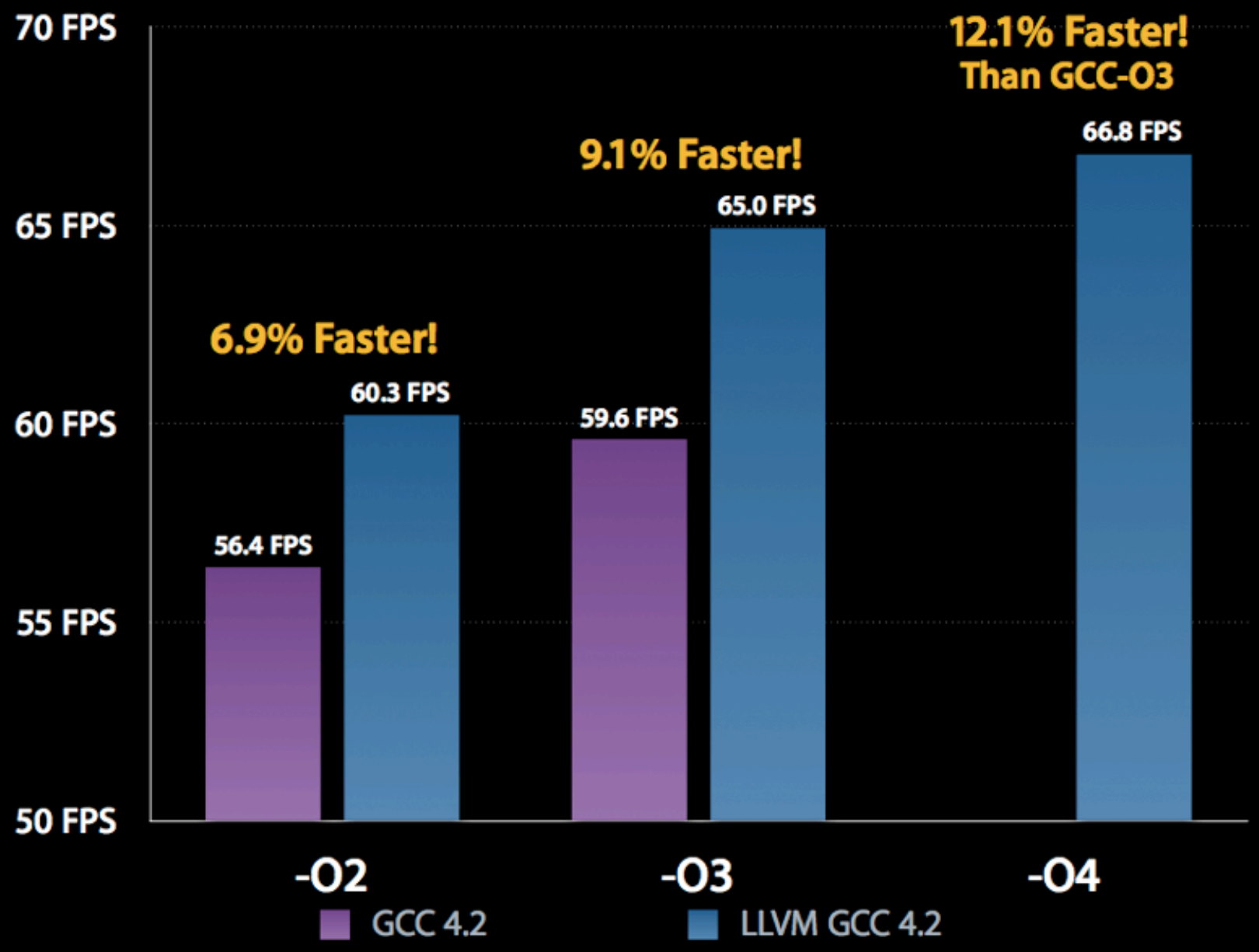
- Generated code
  - How fast does the code run?
- Compile times
  - How fast can we get code from the compiler?
- New features
  - Link Time Optimization

New Feature - Link Time Optimization

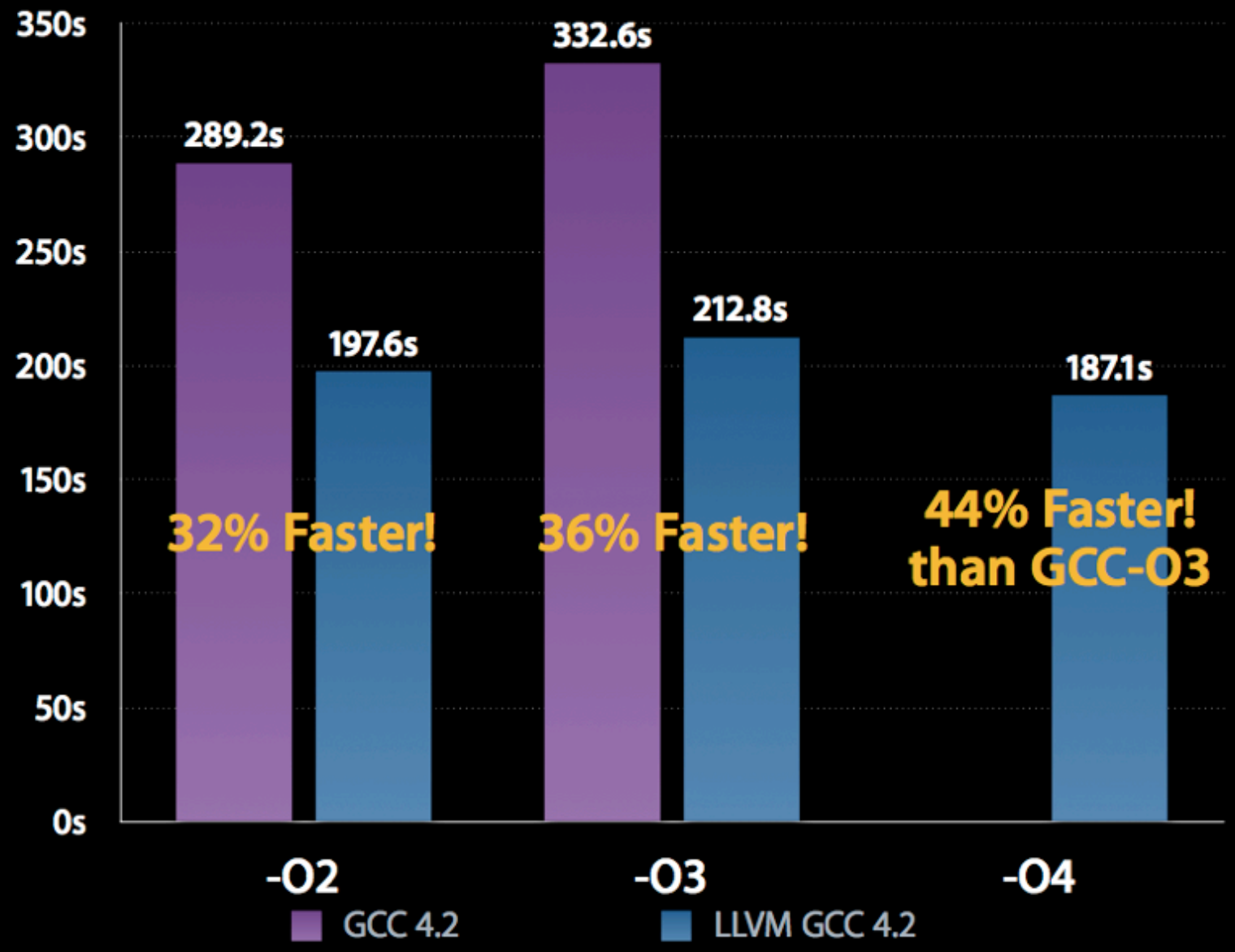
- Optimize (for example: inline, constant fold, and so on) across files with -O4
- Optimize across language boundaries too!



Performance: h.264 Decode Speed



h.264 Compile Times



LLVM-GCC Summary

- Drop in replacement for GCC 4.2
  - Compatible with GCC command line options and languages
  - Works with existing makefiles (e.g. "make CC=llvm-gcc")

- Benefits of LLVM Optimizer and Code Generator
  - Much faster optimizer: ~30-40% at -O3 in most cases
  - Slightly better codegen at a given level: ~5-10% on x86/x86-64
  - Link-Time Optimization at -O4: optimize across source files

- Industrial strength
  - Apple compiles "nightly" with LLVM-GCC its entire MacOS X code base

LLVM for Compiler Hackers

- LLVM is a great target for new languages
  - Well defined, simple to program for
  - Easy to implement new language front-end
  - Easy to retarget existing compiler to use LLVM back-end

- LLVM supports Just-In-Time optimization and compilation
  - Optimize code at runtime based on dynamic information
  - Great for performance, not just for traditional “compilers”

Colorspace Conversion JIT Optimization

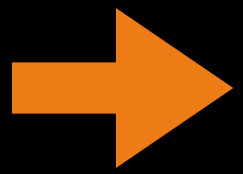
- Code to convert from one color format to another:
  - e.g. BGRA 444R -> RGBA 8888
  - Hundreds of combinations, importance depends on input

```
for each pixel {
  switch (infmt) {
  case RGBA444R:
    R = (*in >> 11) & C
    G = (*in >> 6) & C
    B = (*in >> 1) & C
  ... }
  switch (outfmt) {
  case RGB888:
    *outptr = R << 16 |
              G << 8 ...
  }
}
```

Colorspace Conversion JIT Optimization

- Code to convert from one color format to another:
  - e.g. BGRA 444R -> RGBA 8888
  - Hundreds of combinations, importance depends on input

```
for each pixel {
  switch (infmt) {
  case RGBA444R:
    R = (*in >> 11) & C
    G = (*in >> 6) & C
    B = (*in >> 1) & C
  ... }
  switch (outfmt) {
  case RGB888:
    *outptr = R << 16 |
              G << 8 ...
  }
}
```

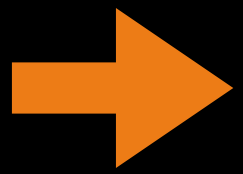


Run-time
specialize

Colorspace Conversion JIT Optimization

- Code to convert from one color format to another:
  - e.g. BGRA 444R -> RGBA 8888
  - Hundreds of combinations, importance depends on input

```
for each pixel {
  switch (infmt) {
  case RGBA444R:
    R = (*in >> 11) & C
    G = (*in >> 6) & C
    B = (*in >> 1) & C
  ... }
  switch (outfmt) {
  case RGB888:
    *outptr = R << 16 |
              G << 8 ...
  }
}
```



Run-time specialize

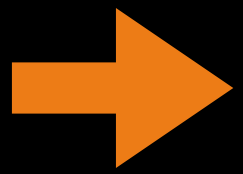
```
for each pixel {
  R = (*in >> 11) & C
  G = (*in >> 6) & C
  B = (*in >> 1) & C
  *outptr = R << 16 |
            G << 8 ...
}
```

Compiler optimizes shifts and masking

Colorspace Conversion JIT Optimization

- Code to convert from one color format to another:
  - e.g. BGRA 444R -> RGBA 8888
  - Hundreds of combinations, importance depends on input

```
for each pixel {
  switch (infmt) {
  case RGBA444R:
    R = (*in >> 11) & C
    G = (*in >> 6) & C
    B = (*in >> 1) & C
  ... }
  switch (outfmt) {
  case RGB888:
    *outptr = R << 16 |
              G << 8 ...
  }
}
```



Run-time specialize

```
for each pixel {
  R = (*in >> 11) & C
  G = (*in >> 6) & C
  B = (*in >> 1) & C
  *outptr = R << 16 |
            G << 8 ...
}
```

Compiler optimizes shifts and masking

Speedup depends on src/dest format:
5.4x speedup on average, 19.3x max speedup (13MB/s to 257MB/s)

LLVM Going Forward

- More of the same...
  - Even faster optimizer
  - Even better optimizations
  - More features for non-C languages
  - Debug Info Improvements
  - Many others...

LLVM Going Forward

- More of the same...
  - Even faster optimizer
  - Even better optimizations
  - More features for non-C languages
  - Debug Info Improvements
  - Many others...

Better tools for source level analysis of C/C++ programs!

Clang

- LLVM native C, Objective-C and C++ front end

- Aggressive project with many goals...
  - Compatibility with GCC
  - Fast compilation
  - Expressive error messages

```
t.c:6:49: error: invalid operands to binary expression ('int' and 'struct A')
  return intArg + func(intArg ? ((someA.X+40) + someA) / 42 : someA.X));
                                ~~~~~~~~~~~~~ ^ ~~~~~
```

- Host for a broad range of source-level tools
  - Indexing, static code analysis, code generation, code completion
  - Source to source tools, refactoring

Clang

- LLVM native C, Objective-C and C++ front end

- Aggressive project with many goals...
  - Compatibility with GCC
  - Fast compilation
  - Expressive error messages

```
t.c:6:49: error: invalid operands to binary expression ('int' and 'struct A')
  return intArg + func(intArg ? ((someA.X+40) + someA) / 42 : someA.X));
                                ~~~~~~~~~~~~~ ^ ~~~~~
```

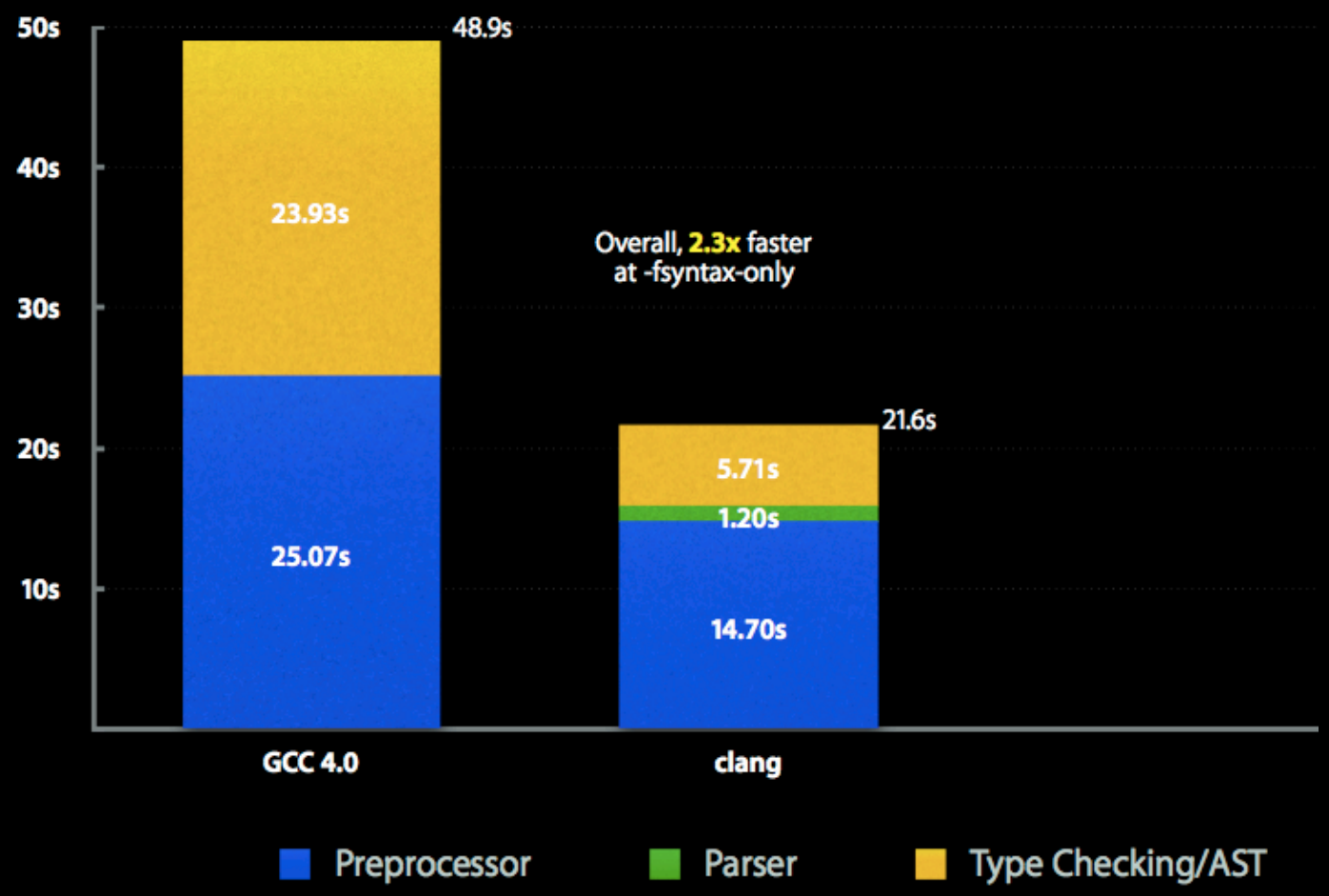
gcc

```
t.c:6:error: invalid operands to binary +
```

- Host for a broad range of source-level tools
  - Indexing, static code analysis, code generation, code completion
  - Source to source tools, refactoring

PostgreSQL Clang Times

- Medium sized C project:
  - 619 C Files in 665K LOC, not counting headers
  - Timings on a fast 2.66Ghz machine, minimum over 5 runs



Conclusions

- New compiler architecture built with reusable components
  - Retarget existing languages to JIT or static compilation
  - Many optimizations and supported targets
- llvm-gcc: drop in GCC-compatible compiler
  - Better & faster optimizer
  - Production quality
- Clang front-end: C/ObjC/C++ front-end
  - Several times faster than GCC
  - Much better end-user features (warnings/errors)
- Ideal set of components to build a C/C++ interpreter with...